Reliability

Problem:

How to maintain

atomicity

durability

properties of transactions

Types of Failures

- Transaction failures
  - Transaction aborts (unilaterally or due to deadlock)
- System (site) failures
  - Failure of processor, main memory, power supply, …
  - Main memory contents are lost, but secondary storage contents are safe
  - Partial vs. total failure
- Media failures
  - Failure of secondary storage devices → stored data is lost
  - Head crash/controller failure
- Communication failures
  - Lost/undeliverable messages
  - Network partitioning

Distributed Reliability Protocols

- Commit protocols
  - How to execute commit command for distributed transactions.
  - Issue: how to ensure atomicity and durability?
- Termination protocols
  - If a failure occurs, how can the remaining operational sites deal with it.
  - Non-blocking: the occurrence of failures should not force the sites to wait until the failure is repaired to terminate the transaction.
- Recovery protocols
  - When a failure occurs, how do the sites where the failure occurred deal with it.
  - Independent: a failed site can determine the outcome of a transaction without having to obtain remote information.
- Independent recovery $\Rightarrow$ non-blocking termination

Two-Phase Commit (2PC)

Phase 1 : The coordinator gets the participants ready to write the results into the database

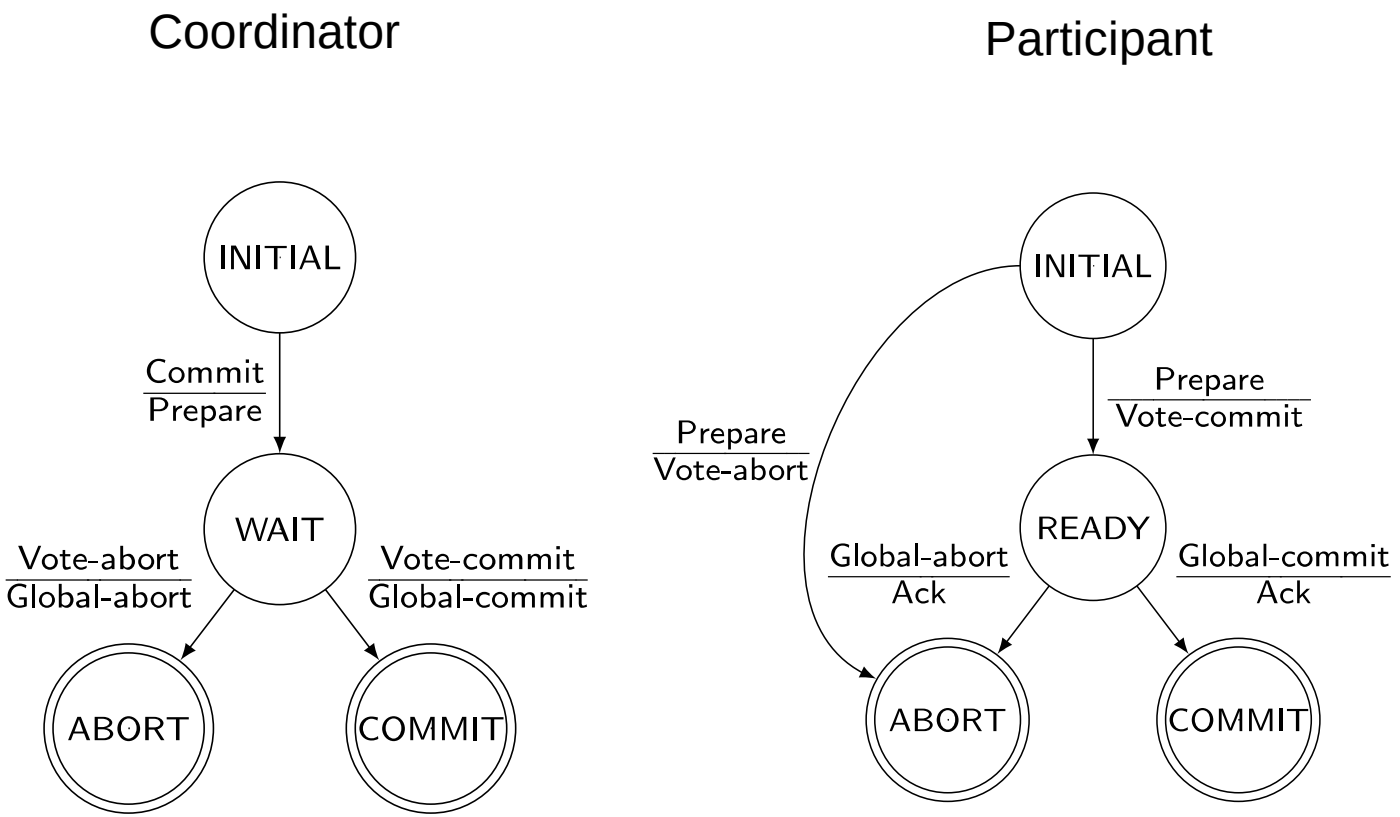
Phase 2 : Everybody writes the results into the database

- **Coordinator** :The process at the site where the transaction originates and which controls the execution
- **Participant** :The process at the other sites that participate in executing the transaction

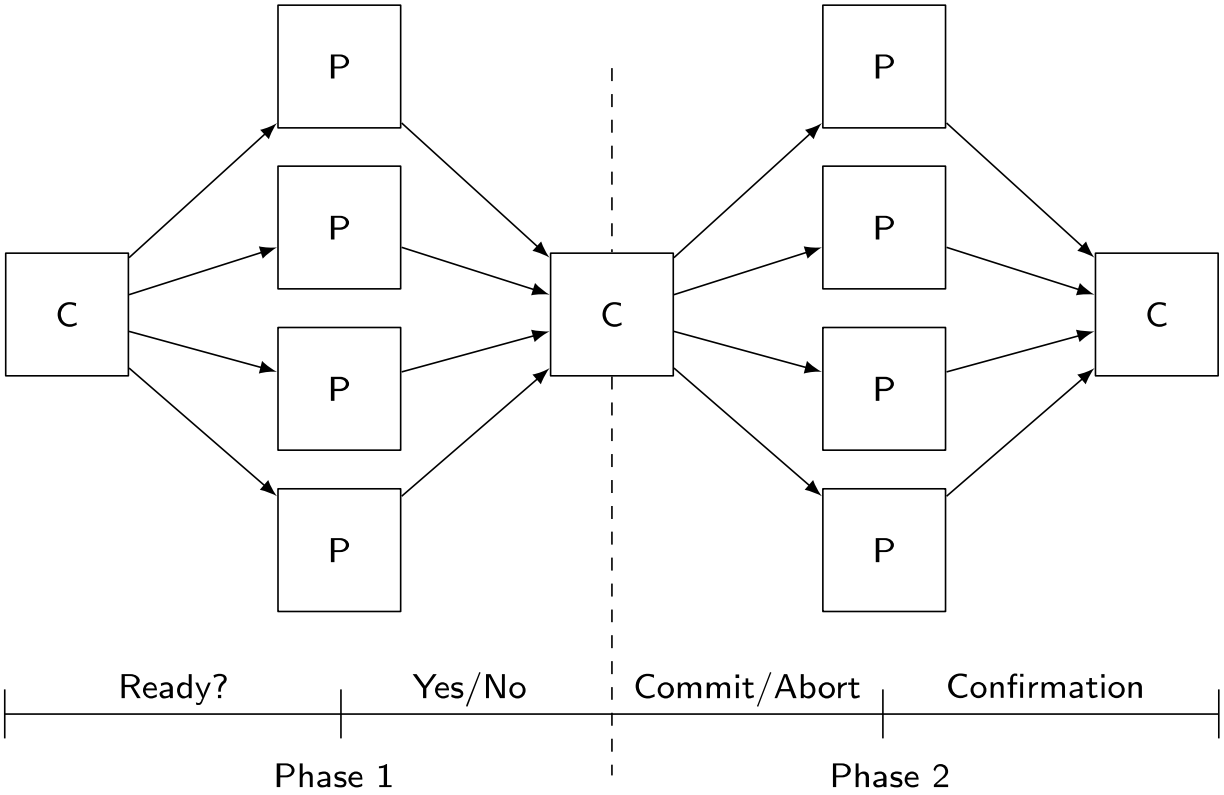
Global Commit Rule:

1. The coordinator aborts a transaction if and only if at least one participant votes to abort it.
2. The coordinator commits a transaction if and only if all of the participants vote to commit it.

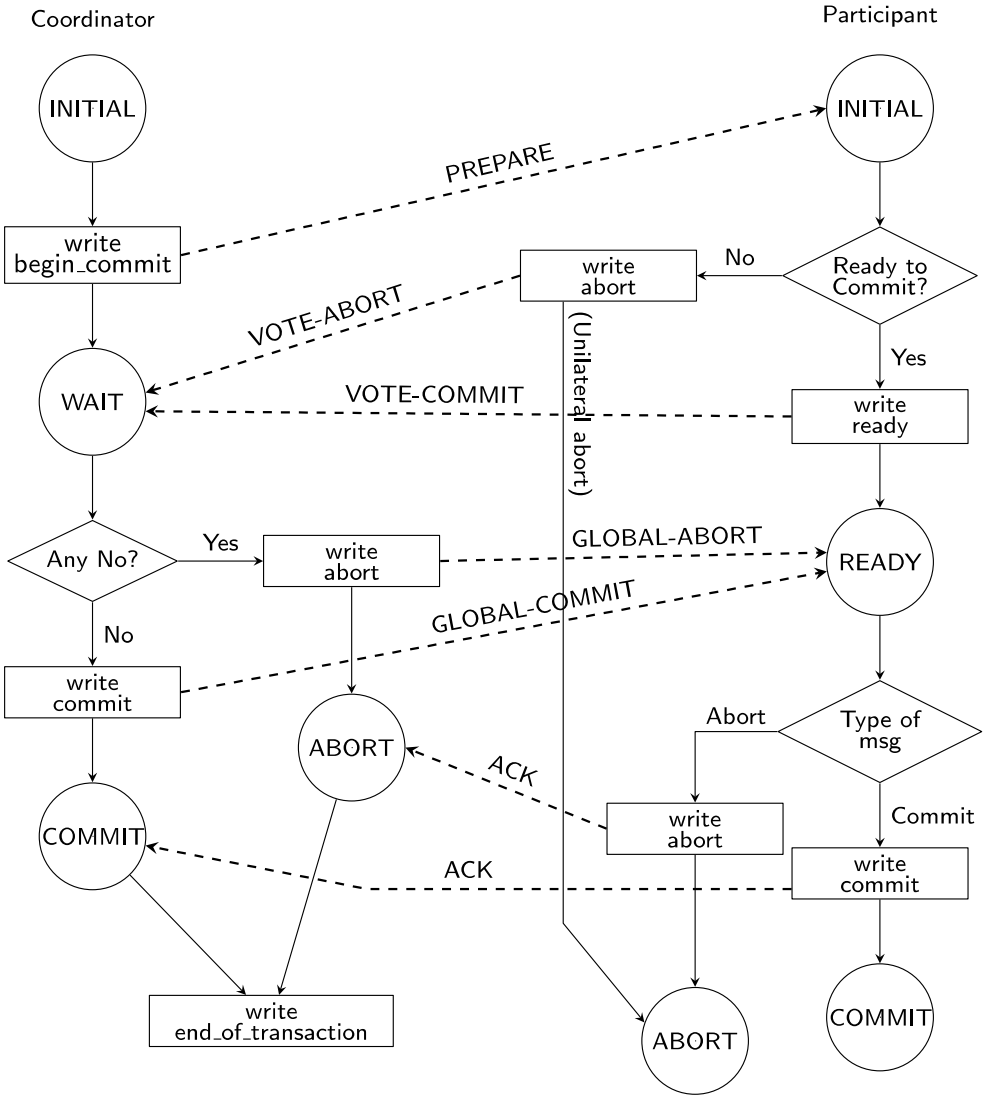
State Transitions in 2PC



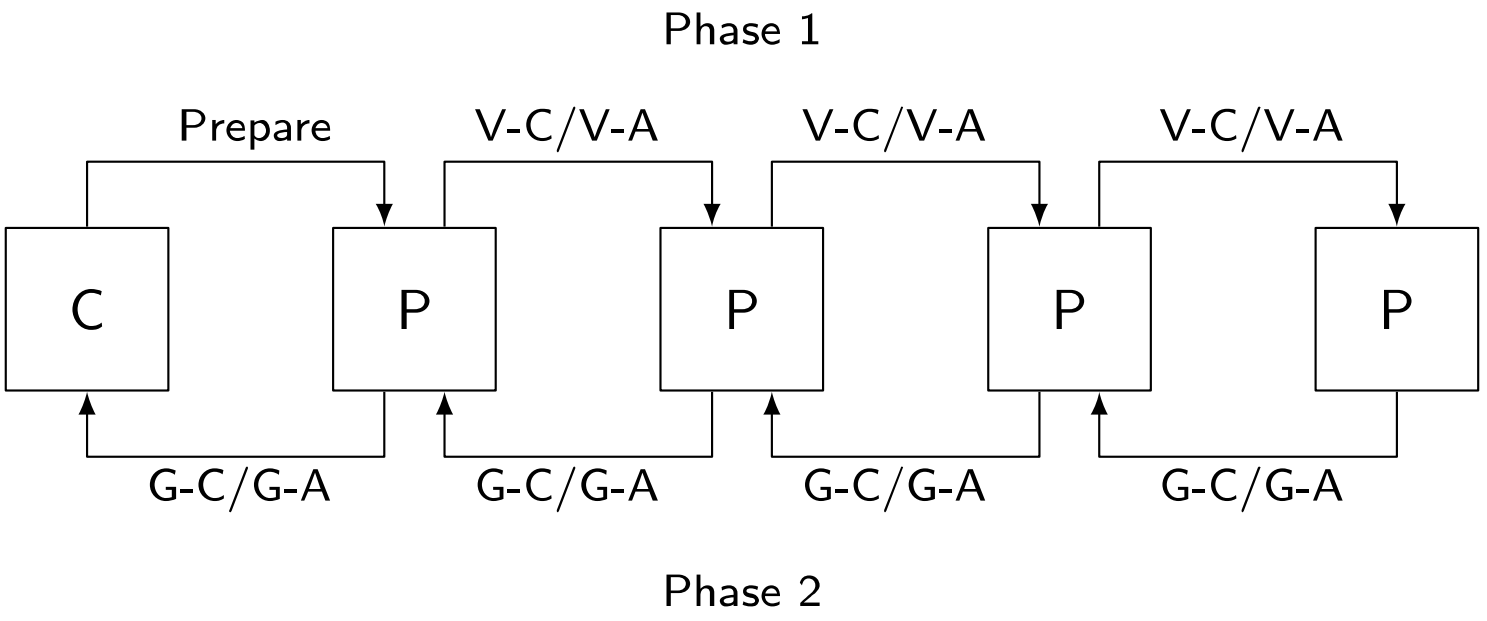
Centralized 2PC



2PC Protocol Actions

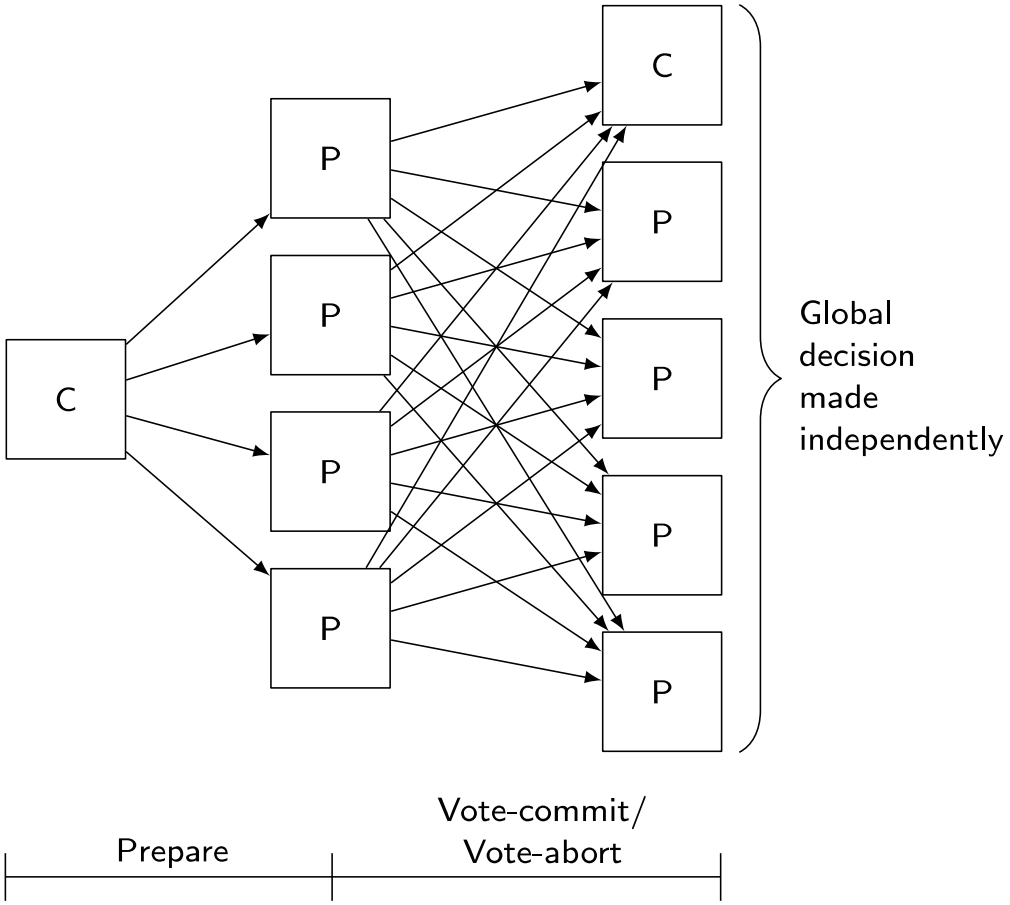


Linear 2PC



V-C: Vote-Commit, V-A: Vote-Abort, G-C: Global-commit, G-A: Global-abort

Distributed 2PC



Variations of 2PC

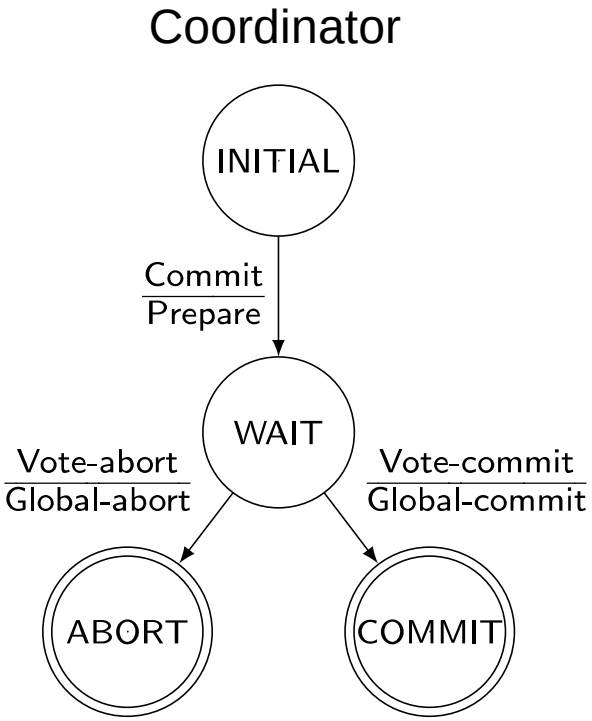
To improve performance by

1) Reduce the number of messages between coordinator & participants
2) Reduce the number of time logs are written

- Presumed Abort 2PC
  - Participant polls coordinator about transaction's outcome
  - No information → abort the transaction
- Presumed Commit 2PC
  - Participant polls coordinator about transaction's outcome
  - No information → assume transaction is committed
  - Not an exact dual of presumed abort 2PC

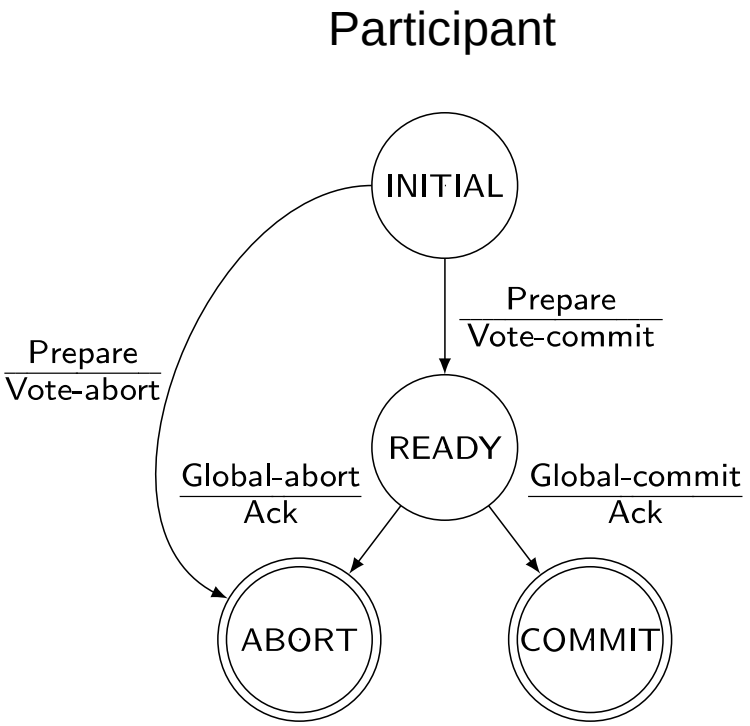
Site Failures - 2PC Termination

- Timeout in INITIAL
  - Who cares
- Timeout in WAIT
  - Cannot unilaterally commit
  - Can unilaterally abort
- Timeout in ABORT or COMMIT
  - Stay blocked and wait for the acks



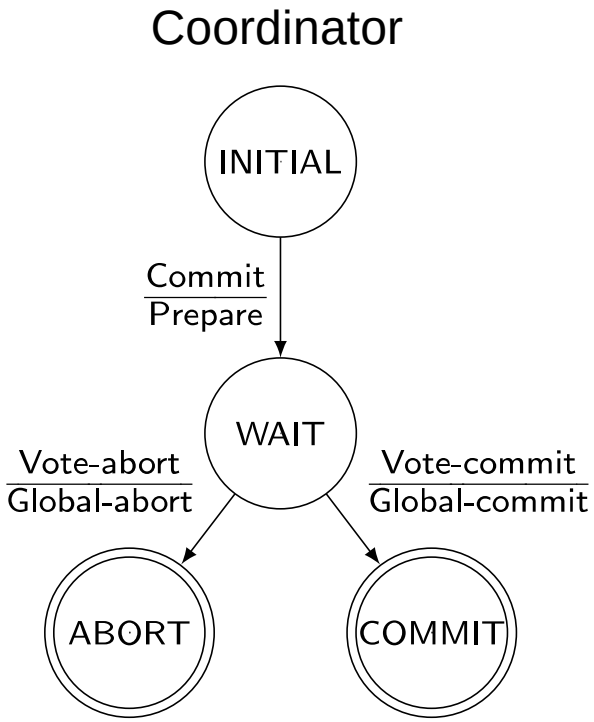
Site Failures - 2PC Termination

- Timeout in INITIAL
  - Coordinator must have failed in INITIAL state
  - Unilaterally abort
- Timeout in READY
  - Stay blocked



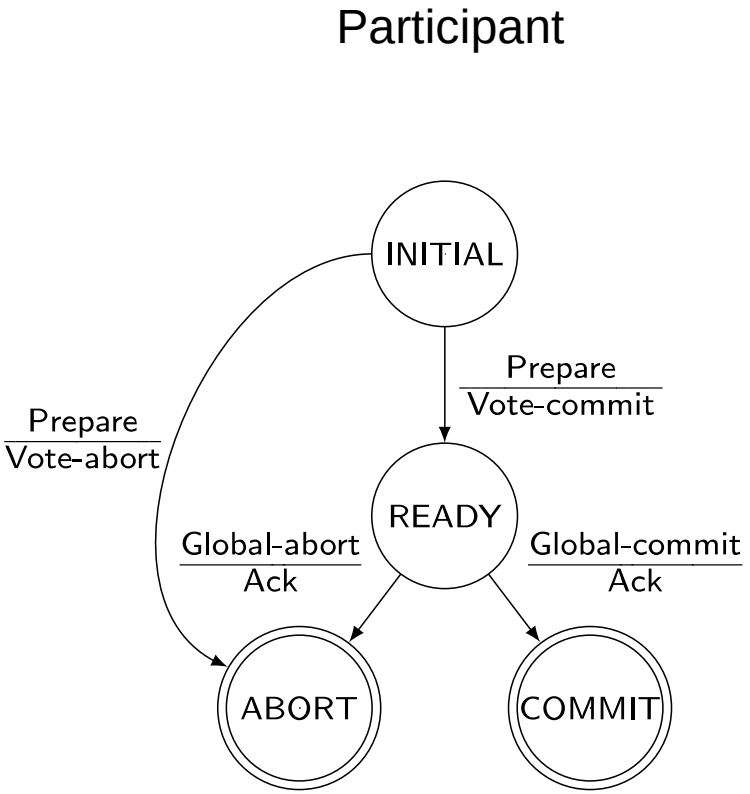
Site Failures - 2PC Recovery

- Failure in INITIAL
  - Start the commit process upon recovery
- Failure in WAIT
  - Restart the commit process upon recovery
- Failure in ABORT or COMMIT
  - Nothing special if all the acks have been received
  - Otherwise the termination protocol is involved



Site Failures - 2PC Recovery

- Failure in INITIAL
  - Unilaterally abort upon recovery
- Failure in READY
  - The coordinator has been informed about the local decision
  - Treat as timeout in READY state and invoke the termination protocol
- Failure in ABORT or COMMIT
  - Nothing special needs to be done



2PC Recovery Protocols – Additional Cases

Arise due to non-atomicity of log and message send actions

- Coordinator site fails after writing “begin_commit” log and before sending “prepare” command
  - treat it as a failure in WAIT state; send “prepare” command
- Participant site fails after writing “ready” record in log but before “vote-commit” is sent
  - treat it as failure in READY state
  - alternatively, can send “vote-commit” upon recovery
- Participant site fails after writing “abort” record in log but before “vote-abort” is sent
  - no need to do anything upon recovery

2PC Recovery Protocols – Additional Case

- Coordinator site fails after logging its final decision record but before sending its decision to the participants
  - coordinator treats it as a failure in COMMIT or ABORT state
  - participants treat it as timeout in the READY state
- Participant site fails after writing “abort” or “commit” record in log but before acknowledgement is sent
  - participant treats it as failure in COMMIT or ABORT state
  - coordinator will handle it by timeout in COMMIT or ABORT state

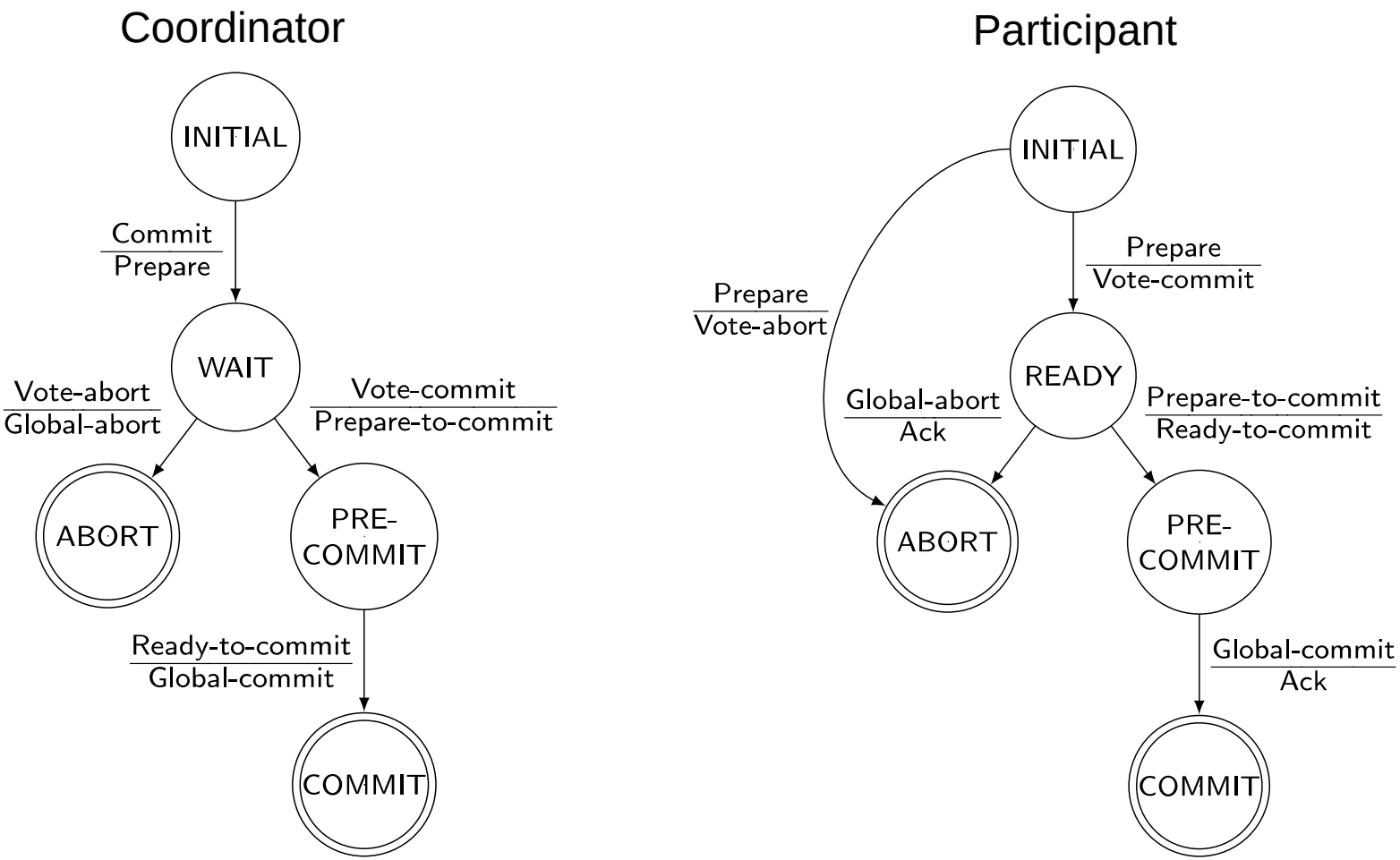
Problem With 2PC

- Blocking
  - Ready implies that the participant waits for the coordinator
  - If coordinator fails, site is blocked until recovery
  - Blocking reduces availability
- Independent recovery is not possible
- However, it is known that:
  - Independent recovery protocols exist only for single site failures; no independent recovery protocol exists which is resilient to multiple-site failures.
- So we search for these protocols – 3PC

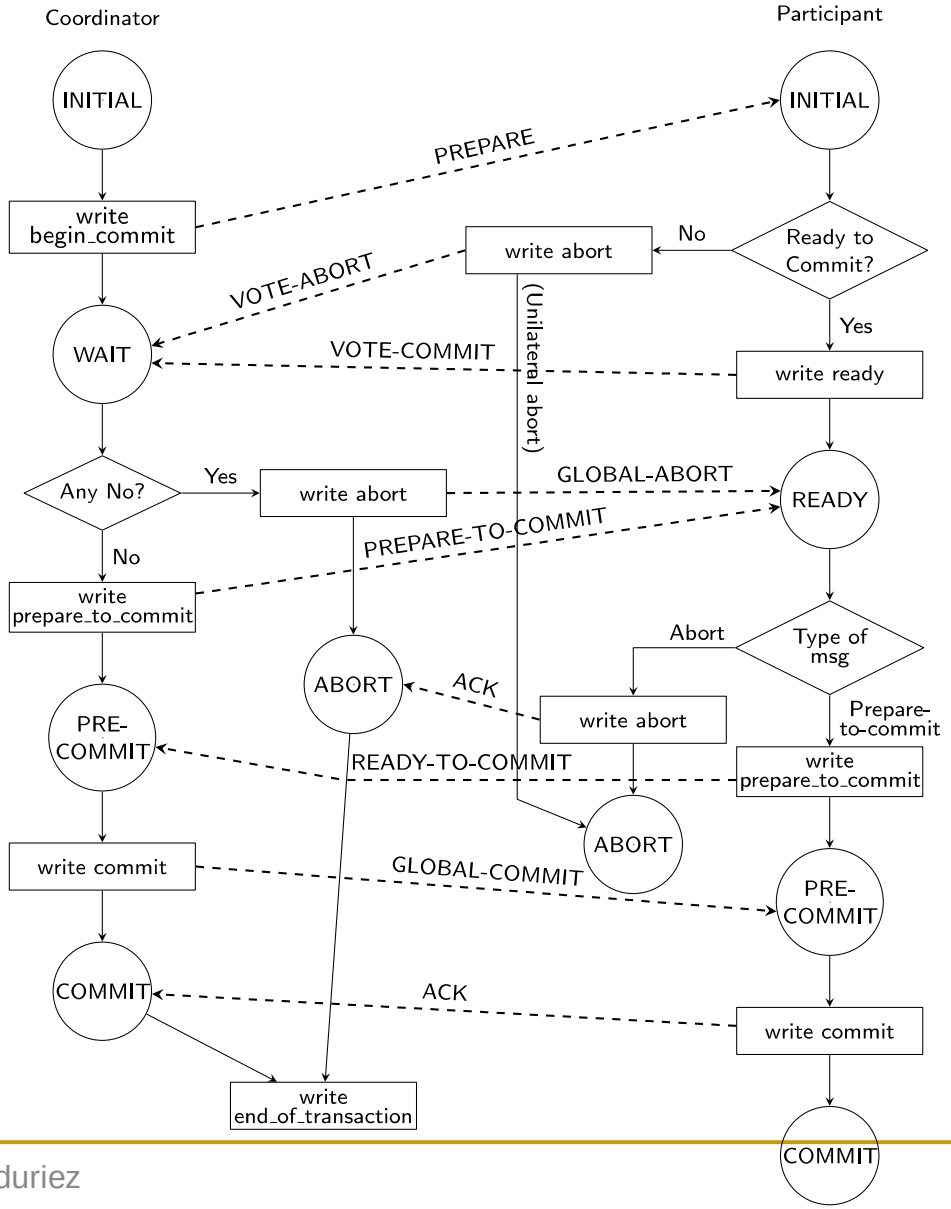
Three-Phase Commit

- 3PC is non-blocking.
- A commit protocols is non-blocking iff
  - it is synchronous within one state transition, and
  - its state transition diagram contains
    - no state which is “adjacent” to both a commit and an abort state, and
    - no non-committable state which is “adjacent” to a commit state
- Adjacent: possible to go from one stat to another with a single state transition
- Committable: all sites have voted to commit a transaction
  - e.g.: COMMIT state

State Transitions in 3PC



3PC Protocol Actions



Network Partitioning

- Simple partitioning
  - Only two partitions
- Multiple partitioning
  - More than two partitions
- Formal bounds:
  - There exists no non-blocking protocol that is resilient to a network partition if messages are lost when partition occurs.
  - There exist non-blocking protocols which are resilient to a single network partition if all undeliverable messages are returned to sender.
  - There exists no non-blocking protocol which is resilient to a multiple partition.

Independent Recovery Protocols for Network Partitioning

- No general solution possible
  - allow one group to terminate while the other is blocked
  - improve availability
- How to determine which group to proceed?
  - The group with a majority
- How does a group know if it has majority?
  - Centralized
    - Whichever partitions contains the central site should terminate the transaction
  - Voting-based (quorum)

Quorum Protocols

- The network partitioning problem is handled by the commit protocol.
- Every site is assigned a vote V_i.
- Total number of votes in the system V
- Abort quorum V_a, commit quorum V_c
  - $V_a + V_c > V$ where $0 \leq V_a , V_c \leq V$
  - Before a transaction commits, it must obtain a commit quorum V_c
  - Before a transaction aborts, it must obtain an abort quorum V_a

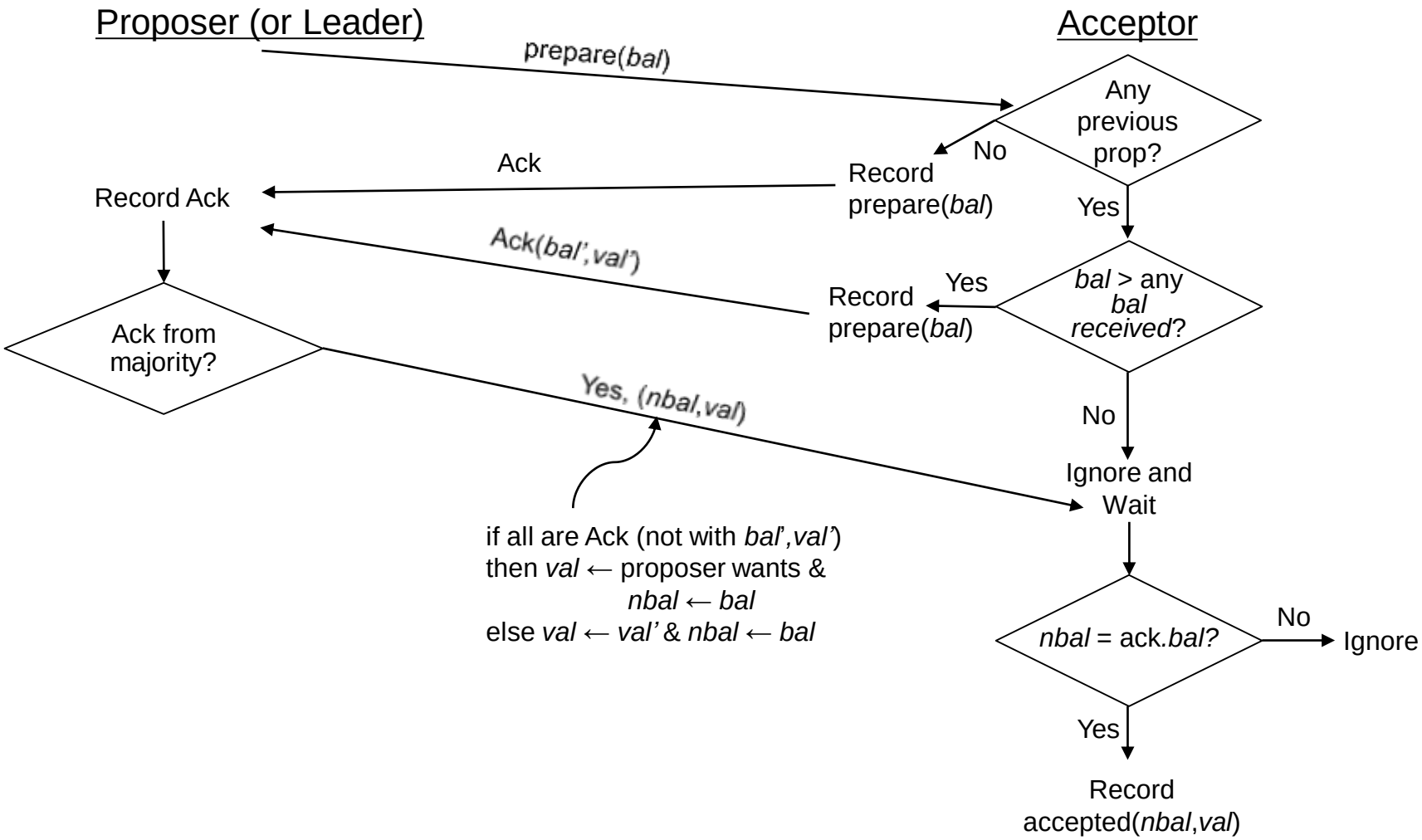
Paxos Consensus Protocol

- General problem: how to reach an agreement (consensus) among TMs about the fate of a transaction
  - 2PC and 3PC are special cases
- General idea: If a majority reaches a decision, the global decision is reached (like voting)
- Roles:
  - Proposer: recommends a decision
  - Acceptor: decides whether to accept the proposed decision
  - Learner: discovers the agreed-upon decision by asking or it is pushed

Paxos & Complications

- Naïve Paxos: one proposer
  - Operates like a 2PC
- Complications
  - Multiple proposers can exist at the same time; acceptor has to choose
    - Attach a ballot number
  - Multiple proposals may result in split votes with no majority
    - Run multiple consensus rounds → performance implication
    - Choose a leader
  - Some accepts fail after they accept a decision; the remaining acceptors may not constitute majority
    - Use ballot numbers

Basic Paxos – No Failures



Basic Paxos with Failures

- Some acceptors fail but there is quorum
  - Not a problem
- Enough acceptors fail to eliminate quorum
  - Run a new ballot
- Proposer/leader fails
  - Choose a new leader and start a new ballot